# RLESS: A Lightweight and Low-Latency Run Length Encoding-Based Spectrum Sensing Algorithm Implemented on RFSoC

Johnny Zhang, Tianyi Zhao, Benjamin W. Domae, Connor Steigerwald, and Danijela Cabric
Electrical and Computer Engineering Department, University of California, Los Angeles, USA
Email: jzhang566@g.ucla.edu, zhaotianyi@ucla.edu,
bdomae@ucla.edu, csteigerwald1@ucla.edu, danijela@ee.ucla.edu

***Abstract*—Wideband spectrum sensing is critical for wireless security, signal intelligence and spectrum sharing systems. In this work, we propose RLESS (Run Length Encoding-based Spectrum Sensor), an algorithm that performs energy detection on individual power spectral densities and stitches detections together across time using run-length encoding. This enables computationally efficient, low-latency labeling without buffering large amounts of data. We provide a real-time implementation of the algorithm on a Zynq Ultrascale+ RFSoC 4x2 kit to generate bounding boxes around transmissions, and evaluate RLESS against similar spectrum sensing algorithms, demonstrating its high accuracy and resource efficiency.**

***Index Terms*—Digital signal processing, wideband spectrum sensing, connected component labeling.**

## I. Introduction

Detecting, localizing, and characterizing unknown radio-frequency emissions in real-time is a fundamental requirement for modern electronic warfare and signal intelligence systems. In congested environments, operators must identify new transmitters, frequency-hopping signals and short-duration bursts within milliseconds to support interference mitigation or electronic attacks.

Traditional spectrum sensing methods typically perform energy detection in the frequency domain, outputting binary occupancy status for each frequency bin with no mechanism to group transmissions across time and compute bounding boxes. Existing methods that do output time-frequency bounding boxes include WHIRLS [1], which iteratively refines boxes via alternating frequency- and time-domain detection, and image-based methods like Searchlight [2], which convolves a channogram image with 2D time-frequency kernels to determine which boxes fit best. While these methods are accurate, they often require many expensive computations over multiple passes through the data. Additionally, image-based methods require an entire image to be buffered before assigning labels, increasing memory requirements and latency.

In this work, we propose RLESS (Run Length Encoding-based Spectrum Sensor), a lightweight and low-latency algorithm designed to address the drawbacks of the aforementioned methods. RLESS performs energy detection on individual power spectral density (PSD) plots, then uses a connected component labeling algorithm based on run length encoding [3] to stitch detections together across PSDs, generating time-frequency bounding boxes. This approach allows for computationally cheap, low-latency detection of transmissions without buffering massive spectrograms. We also produced a real-time implementation of the algorithm on a Zynq Ultrascale+ RFSoC 4x2 kit with the PYNQ platform, resulting in a resource-efficient detector that can report bounding boxes for new transmissions in under a millisecond of their first appearance, while maintaining similar accuracy to more complex algorithms.

## II. Approach

The RLESS algorithm works by converting the incoming I/Q stream into a stream of PSDs. Then, for each PSD, the noise statistics are estimated and used to threshold the PSD into a binary mask. Finally, the run-length encoding algorithm is applied to the stream of binary masks, generating bounding boxes. In this architecture, only one PSD is buffered at a time, leading to low memory usage and latency.

The first stage of the algorithm is the processing of the raw I/Q samples. The analog to digital converters on the RFSoC 4x2 FPGA are connected directly to the programmable logic fabric, and thus minimize latency compared to sending the data through PCIe or Ethernet. The ADC samples at a given sampling rate $f_s$, and has built-in decimation factors $D$ of up to 40x. A Blackman-Harris window is applied to the samples in order to minimize spectral leakage, and the windowed samples are fed into an $N$-point FFT. Every $k$ outputs of $N$ FFT frames are then averaged into a single PSD.

The second stage of the algorithm is noise estimation using a trimmed mean method inspired by TM-CFAR [4]. Here, instead of time-domain estimation from the original paper, we perform the trimmed mean in the frequency domain, as we can only access one PSD at a time. The PSD is split into $S$ buckets and the average power is calculated for each bucket. If it is assumed that the signals are sparse, then buckets with lower average power are less likely to contain any transmissions. Thus, the bucket with the lowest average power is used to estimate the noise floor power $P_n$. Additionally, if

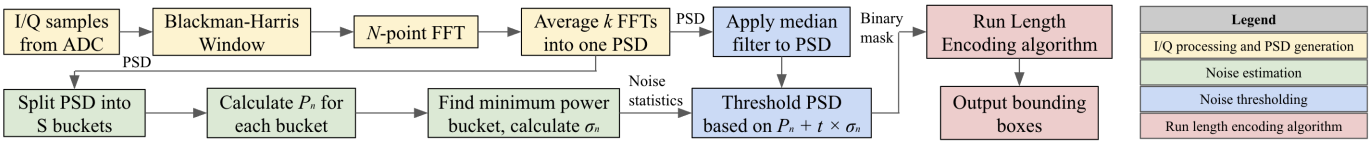



Fig. 1: Flowchart of the RLESS algorithm.

the noise is assumed to be Gaussian, then we can use the standard deviation $\sigma_n$ of the minimum bucket to perform noise thresholding.

The third stage of the algorithm is to threshold the PSD into a binary mask for labeling. A median filter of width $m$ is applied to the spectrogram, iterating through each cell in the PSD and setting its value equal to the median of its $m-1$ neighbors. This filter smooths out noise without blurring the edges of signals, unlike moving average filters. Then, for each cell in the filtered spectrogram, if its power is greater than $P_n + t \times \sigma_n$, where $t$ is a predetermined threshold, the cell is marked as a detection, otherwise it is marked as noise, generating a binary mask.

Now that we have the frequency ranges of detections in each PSD, we must connect them across time to create bounding boxes. To do this, we adapted a run length encoding based blob detection algorithm for video processing [3]. This algorithm operates row-by-row without needing to revisit previous rows, and thus only needs to buffer one row (PSD) at a time instead of buffering an entire image. This fits perfectly into our PSD-streaming architecture. In this algorithm, a list of active transmissions is maintained, consisting of each transmission's start frequency, end frequency and duration. When a PSD is received, the start and end frequencies of each contiguous "run" of detections in the PSD is matched against the list of active transmissions, and the list is updated based on the matches. An optional parameter $r$ can be tuned to set the minimum required length of a run, filtering out small false detections due to noise spikes.

**Algorithm 1** Run Length Encoding Step

**Require:** Active component list $A_i = (f_S, f_E, t)$, list of contiguous runs $R_i = (f_S, f_E)$
**Ensure:** $A$ updated, active and closed components reported
$A' \leftarrow \emptyset$
$D_A \leftarrow$ boolean array of size $|A|$, initialized to 0
**for** each run $r \in R$ **do**
  $C \leftarrow (r.f_S, r.f_E, 1)$
  **for** each $c_i \in A$ **s.t.** $r.f_S < c_i.f_E$ **&** $r.f_E > c_i.f_S$ **do**
    $C \leftarrow C \cup (c_i.f_E, c_i.f_S, c_i.t + 1)$, $D_A[i] = 1$
  **end for**
  **if** $|C| = 1$ **then**
    $A' \leftarrow A' \cup C$
  **else if** $|C| > 1$ **then**
    $A' \leftarrow A' \cup (min(C.f_S), max(C.f_E), max(C.t))$
  **end if**
**end for**
**for** each $c_i \in A$ **do**
  $D_A[i] = 1$ **?** $c_i$ is active **:** $c_i$ is closed
**end for**
$A \leftarrow A'$

The computational complexity of the RLESS algorithm is dominated by the PSD accumulation step, which has a time complexity of $O(kNlogN)$. The other steps in the pipeline have a complexity closer to $O(N)$. In terms of latency, the latency between receiving I/Q samples from the ADC and outputting bounding boxes is also dominated by the PSD accumulation step; the latency of the Blackman-Harris window, FFT, noise estimation and run-length encoding algorithm are all much lower. In our FPGA implementation, we achieved an average latency of 39,752 clock cycles, or 0.326 milliseconds. It can also be shown that the frequency resolution is $f_s/DN$, and the time resolution is equal to the time duration of a PSD, or $NkD/f_s$.

The parameters $k$, $t$ and $r$ are adjustable at runtime, and can be tuned to suit the needs of particular implementations. Since the latency and time resolution are both proportional to $k$, smaller values of $k$ result in finer time resolution and lower latency, with the tradeoff of introducing more noise. Increasing $t$ decreases the false alarm rate, but also decreases the detection rate. A larger value of $r$ makes the algorithm more robust to false detections, but can also cause narrowband signals to be filtered out. The selection of parameters is further discussed in Section III.

## III. Evaluation

We evaluate RLESS both quantitatively and qualitatively. First, we simulated the algorithm in MATLAB and tested on a synthetic dataset with known time-frequency labels, containing random QPSK signals of bandwidths ranging from 1-10 MHz. We compare RLESS to both Searchlight [2] and WHIRLS [1] as baselines. Following the WHIRLS paper, we determine accuracy based on the percentage of detections, false detection rate, and the root mean square error of the time and frequency bounds. Then, we perform qualitative evaluations on real over-the-air RF signals, where an antenna was connected to the RFSoC's ADC and used to capture signals in the 2.4 GHz band from around the room. All of the MATLAB simulation and FPGA implementation code will be released to Github.

TABLE I: RLESS parameters.

| Variation | $k$ | $m$ | $t$ | $r$ | $N$ | $f_s$ | $D$ |
|---|---|---|---|---|---|---|---|
| RLESS sim. 1 | 16 | 15 | 3 | 12 | 1024 | 50 MHz | 0 |
| RLESS sim. 2 | 4 | 21 | 3.5 | 6 | 1024 | 50 MHz | 0 |
| Real-time | 16 | 7 | 3 | 20 | 1024 | 4.9152 GSPS | 40 |

The parameters of RLESS used for the simulation and the real-time implementation are shown in Table I. The values of $k$, $m$, $t$ and $r$ used for RLESS simulation 1 allow for effective

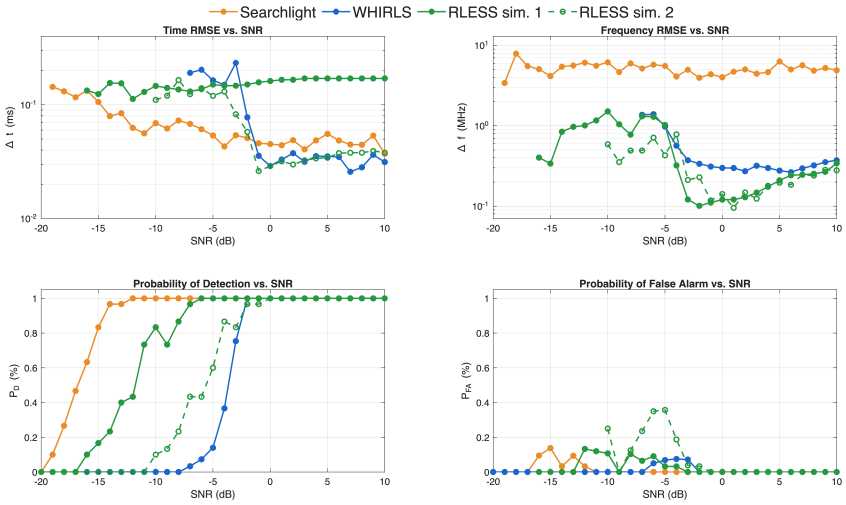



Fig. 2: Accuracy metrics for RLESS, Searchlight and WHIRLS.

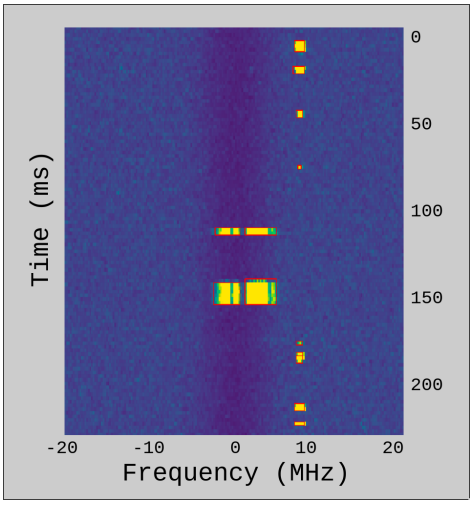



Fig. 3: Real-time RLESS results.

smoothing of noise and removal of false detections without filtering out too many true detections. As can be seen from Figure 2, RLESS simulation 1 has slightly worse probability of detection and time RMSE than Searchlight, but much better frequency RMSE. Compared to WHIRLS, RLESS actually achieves better probability of detection and frequency RMSE, though unlike WHIRLS, RLESS does not support independent time and frequency resolutions. Additionally, if two transmissions overlap, RLESS is unable to distinguish between them and will label them as a single detection.

When comparing the two implementations of RLESS, simulation 2 with $k = 4$ features better time RMSE and lower latency, at the cost of lower probability of detection and higher false alarm rate. Because the value of $k$ is lower, there is less PSD averaging, leading to more noise; thus, the threshold and the width of the median filter must be increased in order to deal with the increase of false detections.

A visualization of RLESS real-time RFSoC implementation is shown in Figure 3. The figure shows the output of the waterfall display, where two 10 MHz transmissions can be seen, along with a few smaller 1-2 MHz signals. RLESS correctly draws bounding boxes around all these transmissions, demonstrating its effectiveness in real-world, over-the-air scenarios. In terms of resource utilization, the PSD accumulation, noise estimation, noise thresholding and run length encoding steps used 25 block RAMs, 78 DSP slices, 11,457 flip flops and 20,997 lookup tables. These take up less than 3% of the RFSoC's total resources.

## IV. Demo Plan

The demonstration will showcase the real-time RLESS pipeline running live on the Zynq Ultrascale+ RFSoC 4x2 platform. Two signal sources will be used to test the system: first, we will inject pre-recorded transmissions with known bounding boxes from the onboard DAC into the ADC via an SMA cable. Then, we will use an antenna to listen to the live RF environment of the room itself, labeling transmissions from other devices and demonstrations. This combination allows the audience to observe controlled test cases that highlight the algorithm's behavior under specific conditions, as well as unpredictable, real-world signals.

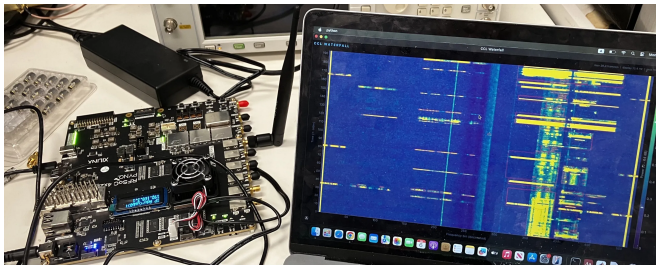


Fig. 4: Setup with RFSoC 4x2 and waterfall display.

The audience-facing display will consist of a live-updating waterfall plot showing the incoming spectrum, with bounding boxes drawn around transmissions in real-time as RLESS identifies them. There will also be a metrics panel reporting live statistics, such as latency and running counts of active transmissions. Throughout the demo, we will vary the injected recorded data and invite attendees to introduce their own RF sources to test detection of unplanned signals. We will also change the run-time parameters $k$, $t$ and $r$, and discuss their effect on latency and accuracy.

## References


[1] T. Zhao, B. W. Domae, C. Steigerwald, L. B. Paradis, T. Chabuk, and D. Cabric, "Whirls: Wideband high-resolution labeling of signals for spectrum sensing applications," in *MILCOM 2025 - 2025 IEEE Military Communications Conference (MILCOM)*, pp. 323–329, 2025.

[2] R. Bell, K. Watson, T. Hu, I. Poy, F. Harris, and D. Bharadia, "Searchlight: An accurate, sensitive, and fast radio frequency energy detection system," in *MILCOM 2023 - 2023 IEEE Military Communications Conference (MILCOM)*, pp. 397–404, 2023.

[3] J. Trein, A. T. Schwarzbacher, and B. Hoppe, "Fpga implementation of a single pass real-time blob analysis using run length encoding," in *MPC-Workshop. Ravensburg-Weingarten*, pp. 71–77, Germany USA, 2008.

[4] M. A. Richards, J. A. Scheer, and W. A. Holm, "Constant False Alarm Rate Detectors," in *Principles of Modern Radar: Basic Principles*, ch. 16, pp. 589–620, SciTech Publishing, Inc., 2010.